Московский физико-технический институт
Высшая школа программной инженерии

Формальные языки и сложность вычислений
3 семестр, 2026/2027 уч. год

Лекция 4
Задача SAT. NP-полнота задач SAT, 3-SAT.

Лектор: Зухба Анастасия Викторовна
Автор: Опоздун

1 Разминочка

Определение 1.1. *Язык A называют* NP-*трудным или* NP-*hard, если* $\forall L \in \mathrm{NP} \hookrightarrow L \leq_p A$.

Определение 1.2. *Язык A называют* NP-*полным или* NPC, *если* $A \in \mathrm{NP}$ *и* $\forall L \in \mathrm{NP} \hookrightarrow L \leq_p A$.

Замечание 1.1. *Теперь зададимся вопросом: а существует ли вообще* NP-*полный язык? Оказывается, существует. Возьмем в качестве примера такого языка следующее чудо:*

$$\mathrm{MSAT} = \left\{ < N >,\ x,\ 1^{P(|x|)} \middle| \exists y : N(x,y) = 1 \text{ и время работы } N(x,y) \leq P(|x|) \right\}$$

здесь N — некоторая машина Тьюринга, $< N >$ — её кодировка. По сути этот язык состоит из машины Тьюринга $\mathbf{N}$, слова $\mathbf{x}$ и времени работы $\mathbf{1^{P(|x|)}}$. Заметим, что $\mathrm{MSAT} \in \mathrm{NP}$*: в самом деле, в качестве верификатора мы можем взять*

$$V\left(\left(< N >,\ x,\ 1^{P(|x|)}\right), y\right)$$

который просто эмулирует $N(x,y)$. Теперь возьмем произвольный язык $L \in \mathrm{NP}$. Для этого языка существует N_L — верификатор, у которого свой полином $P_L(|x|)$. Тогда мы можем сопоставить

$$x \xrightarrow{f(x)} \left(< N_L >, x, 1^{P_L(|x|)}\right)$$

$f(x)$ вычислима за полином, так как N_L — константа, x — линия, $1^{P_L(|x|)}$ — конструируемая по времени функция, то есть тоже полиномчик. Вуаля, у нас есть сводимость по Карпу. Отсюда получаем, что

$$x \in L \Longleftrightarrow \left(< N_L >, x, 1^{P_L(|x|)}\right) \in \mathrm{MSAT} \tag{1}$$

Тут важно отметить, что вместе с языком L мы фиксируем и конкретный верификатор N_L, и именно для этой пары справедливо (1). А так в нашем языке MSAT *могут лежать совсем не верификаторы какого-либо языка.*
Также конструкция $\left(< N >,\ x,\ 1^{P(|x|)}\right)$, дает нам возможность построить МТ N' работающую ровно за $P(|x|)$, то есть, этот полином мы можем использовать не только для проверки, но и для принудительной остановки.

2 NP-полнота задачи SAT

Определение 2.1.

$$\mathrm{SAT} = \{\varphi \mid \varphi \text{ — выполнимая булева формула}\}$$

Замечание 2.1. *Если $A \leq_p B$ и при этом $A \in$* NPC, *то $B \in$* NP-*hard (в силу транзитивности). В качестве задачи A отлично подходит наша задача* SAT.

Определение 2.2.

$$\text{3-SAT} = \{\varphi \mid \varphi \text{ — выполнимая булева формула в виде 3-КНФ}\}$$

Теорема 2.1 (Кука-Левина)**.**

$$\mathrm{SAT} \in \mathrm{NPC}$$

Доказательство. Начнем с доказательства принадлежности SAT классу NP: это легко следует из сертификатного определения класса NP, ведь нам достаточно в качестве сертификата взять набор значений переменных, то есть, выполняющий набор. А его подстановка в формулу происходит за полиномиальное время.

Теперь возьмем произвольный язык $L \in \mathrm{NP}$ и построим всюду определённую, полиномиально вычислимую функцию f, принимающую на вход слово x, принадлежность которого языку L мы хотим проверить, а на выход отдающую некоторую формулу:

$$x \in L \iff f(x) \in \mathrm{SAT}$$

То есть, слово x принадлежит языку L тогда и только тогда, когда $f(x)$ — вычислимая формула.
Задавать произвольный язык $L \in \mathrm{NP}$ будем его верификатором. Напомним, что верификатор — это ДМТ с двумя аргументами $V(x, s)$, работающая за полином от первого $P(|x|)$, такая что

$$x \in L \iff \exists s : \ V(x, s) = 1$$

$$x \notin L \iff \forall s \hookrightarrow V(x, s) = 0$$

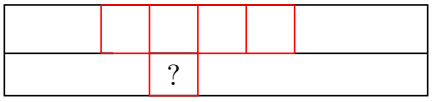
Поскольку всё, что осуществляется за полином на k лентах, можно осуществить за полином и на одной, то пусть наш верификатор будет одноленточным, да ещё и бесконечным в одну сторону (мы помним, что на бесконечном в одну сторону можно выполнить всё, что выполнимо на бесконечном в обе стороны). И чтоб жизнь совсем казалась мёдом, будем считать, что длина сертификата ограничена полиномом $P(|x|)$.
Добавим несколько служебных символов, которые будут обозначать: $\triangleright$ — начало, $\square$ — разделитель между x и s, $\triangleleft$ — конец и $\#$. Мы считаем, что этих служебных символов не было в конечном алфавите верификатора Σ. Пусть Q — конечный набор состояний верификатора.
Представим, что у нас есть последовательность конфигураций с x и удачным, подходящим сертификатом, выглядит это следующим образом:

$$q_0 \triangleright \underbrace{x\square s\#...\#}_{P(|x|)} \triangleleft$$

Поскольку верификатор — это ДМТ, то каждая конфигурация получается из ленты и записи состояния с указанием, где оно находится:

$$\triangleright q_1 x[1] \ldots \square \ldots \# \triangleleft$$

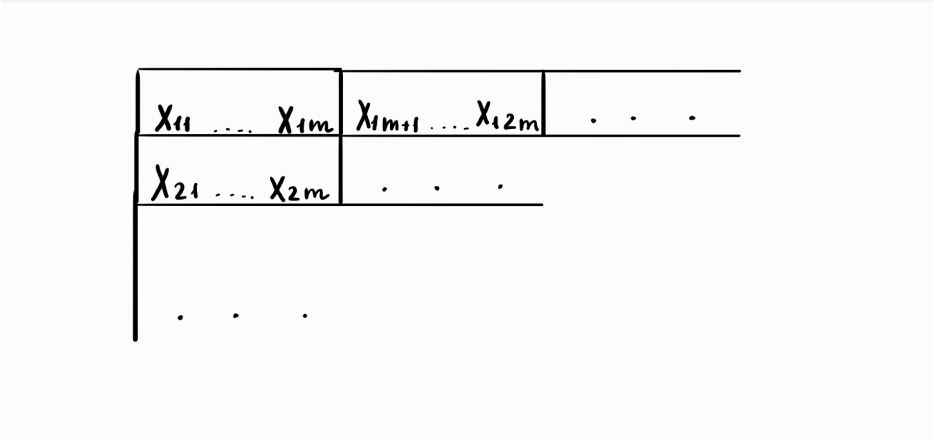
Выпишем последовательно все конфигурации и рассмотрим некоторые две соседние. Пусть мы хотим узнать, какой символ стоит в i-ой ячейке:



Утверждается, что для этого достаточно посмотреть на красную фигурку: в самом деле, если q находится вне верхних четырёх клеточек, то искомая ячейка равна тому, что находится прямо над ней. Если же q находится в первой красной клетке, то следующая может поменяться за один шаг, и тогда нам нужно обратиться к таблице переходов δ. Если q находится во второй клетке, то головка может сдвинуться, следовательно, поменяется содержимое ячейки. Пристально посмотрев на две оставшиеся клеточки, поймём, что они могут нам пригодиться, а вот все остальные нам не нужны и никакой погоды не сделают.
В итоге, чтобы вычислить строку, достаточно прогуляться вот такой красной фигуркой по всей предыдущей строке и t раз (t — длина конфигурации) проверить, можно ли применить правило и что-то найти. На выходе мы получим таблицу, заполненную на основе x и s, и наша задача — проверить корректность этой таблицы. Сделать мы это можем за что-то порядка $O(P(|x|^2))$.
Теперь пусть

$$n = |Q| + |\Sigma| + y, \quad y \text{ — количество служебных символов}$$

найдем m — наименьшее натуральное число, такое что $n \leq 2^m$, тогда m битов нам хватит, чтобы закодировать любой символ этого алфавита. Отметим, что m — константа, зависящая от L, но не от x. Теперь закодируем нашу таблицу конфигураций 0 и 1. И перед нами встаёт задача составить формулу, которая будет выполнима, только если для заданного фиксированного x существует "удачный" сертификат.
Возьмём первую строчку закодированной таблицы, разобьем её на блоки размера m, но вместо 0 и 1 будем записывать переменные:



Теперь по этим переменным будем писать формулу: предположим, что $q_0 = 10011$. Строим φ:

$$\varphi = (x_{11} \equiv 1) \wedge (x_{12} \equiv 0) \wedge (x_{13} \equiv 0) \cdots \wedge (*) \wedge (\square) \wedge (\triangleleft) \wedge (**) \tag{2}$$

$*$ — посимвольная проверка того, что здесь находится x

$**$ — проверка перехода с 1 строки на 2

Проверка перехода на другую строку — это фактически проверка красной доминошки некоторой сложной громоздкой формулой, которая, тем не менее, является константной. Заметим, что проверку сертификата мы не делаем (потому что мы и не знаем, какой сертификат нам нужен). Также мы выбираем такой верификатор, что он стирает все с ленты, записывает на нее 1 и останавливается в состоянии q_a, если сертификат является выполняющим набором. Продолжаем строить формулу:

$$\varphi = (2) \wedge (\text{переход с 2 на 3}) \wedge \cdots \wedge (\triangleright q_a 1 \# \ldots \# \triangleleft)$$

Заметим, что нам не нужно отслеживать, одно ли у нас состояние в каждой строке, потому что верификатор является ДМТ, и никакой неопределённости быть не может.
Итак, мы получили очень большую страшную формулу, при этом она выполнима $\Longleftrightarrow$ протокол корректен, то есть, корректен сертификат, и верификатор на этом сертификате останавливается в принимающем состоянии. Такими вот движениями мы доказали, что произвольный язык $L \in$ NP полиномиально сводится к SAT. Пару слов о полиномиальности: квадратичную таблицу от $P(|x|)$ мы завели за полином, построили формулу тоже за полином (тоже что-то около квадрата). $\square$

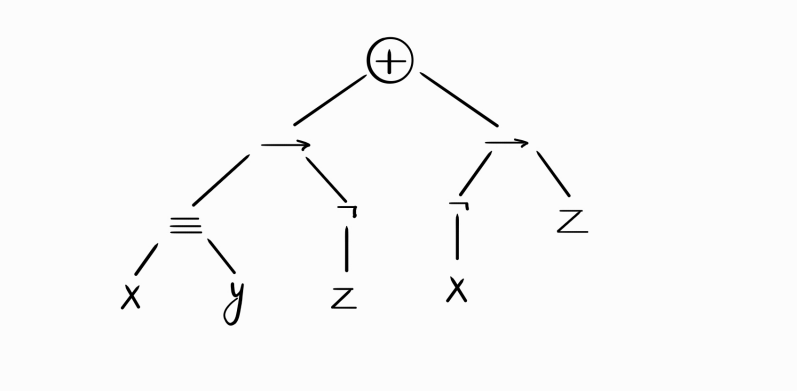
Теорема 2.2.

$$\text{3-SAT} \in \text{NPC}$$

Доказательство. Для доказательство нужно свести SAT $\leq_p$ 3-SAT (обратная сводимость очевидна, потому что 3-SAT — частный случай SAT. К тому же, 3-SAT $\in$ NP опять-таки в силу того, что 3-SAT $\subset$ SAT и SAT $\in$ NP). Пусть у нас есть страшная формула, например

$$\varphi = ((x \equiv y) \to (\neg z)) \oplus ((\neg x) \to z)$$

Нарисуем дерево этой формулы:



Заметим, что каждый символ в формуле, кроме скобок, присутствует в дереве только 1 раз. Значит, у нашей формулы не более чем линейно подформул (каждой подформуле соответствует ветка (поддерево данной вершины), а количество веток равно количеству вершин). Теперь на каждую подформулу, кроме листов, заведём отдельную переменную и будем превращать исходную формулу в 3-КНФ следующим образом: каждой вершине a, имеющей детей b и c будем сопоставлять формулу вида

$$a \equiv (b \ op \ c)$$

А любая такая формула имеет своё 3-КНФ представление, причем в этом представлении не более 8 дизъюнктов. Все такие формулы мы будем объединять через конъюнкции, причём таких конъюнкций наберётся не более, чем линия по длине формулы (потому что у каждой вершины дерева не более чем 8 дизъюнктов по 3 переменные). И не забудем потребовать, чтобы переменная, соответствующая корню дерева, равнялась 1.

Таким образом, мы построили всюду определённую полиномиально вычислимую функцию, которая по произвольной формуле φ строит формулу φ' в виде 3-КНФ, и φ выполнима $\iff \varphi'$ выполнима. То есть, мы свели SAT к 3-SAT $\implies$ 3-SAT $\in$ NPC. $\square$